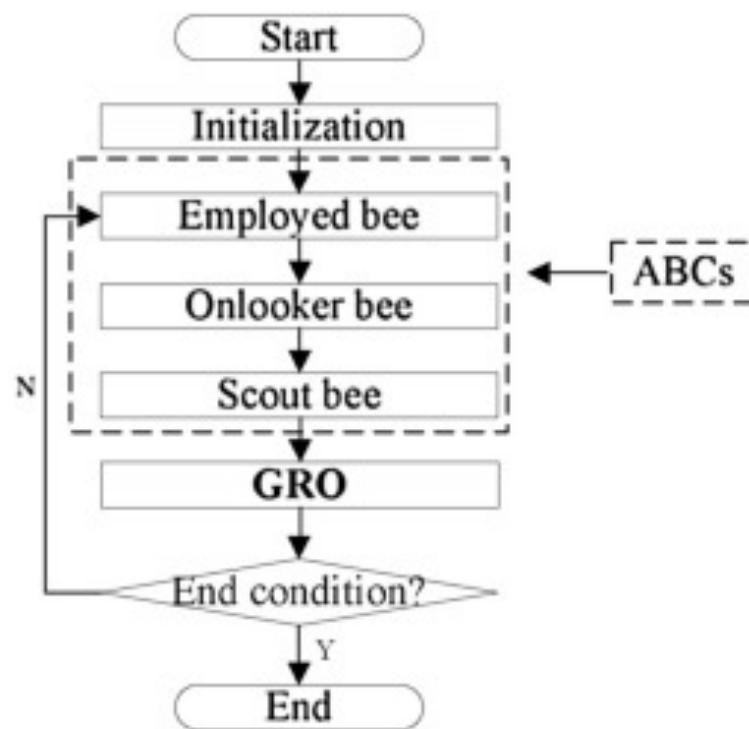


BASIC ARTIFICIAL BEE COLONY (ABC) ALGORITHM IMPLEMENTATION AND SIMULATION



By / Adam Tayel

the Artificial Bee Colony (ABC) algorithm for minimization and maximization of a 4-variable Ackley function. Python plots the convergence, while NetLogo simulates food sources and scout bees.

1) Objective Function :

We implemented the **Ackley Function**, which is a widely accepted non-linear, multi-modal benchmark function for optimization algorithms. The function evaluates **4 decision variables** (x_1, x_2, x_3, x_4) within the search space of $[-5.0, 5.0]$.

2) Model Parameters :

The parameters used for running both the **minimization** and **maximization** versions of the ABC algorithm are configured as follows:

Parameter	Value	Description
Colony Size	30	Total number of bees in the colony
Food Sources	15	Number of active candidate solutions ($\text{Colony Size} / 2$)
Limit	20	Abandonment threshold before a solution is reset by a Scout Bee
Max Iterations	50	Total number of search optimization cycles

3) Python Implementation Results :

1. Optimization Logs (Sample Data)

The Python engine successfully ran 50 iterations for both optimization modes:

- **Minimization Target:** Converged towards the global minimum of \$0.0\$.
- **Maximization Target:** Reached the peak bound inside the bounded constraint matrix.

2. Convergence Plots :

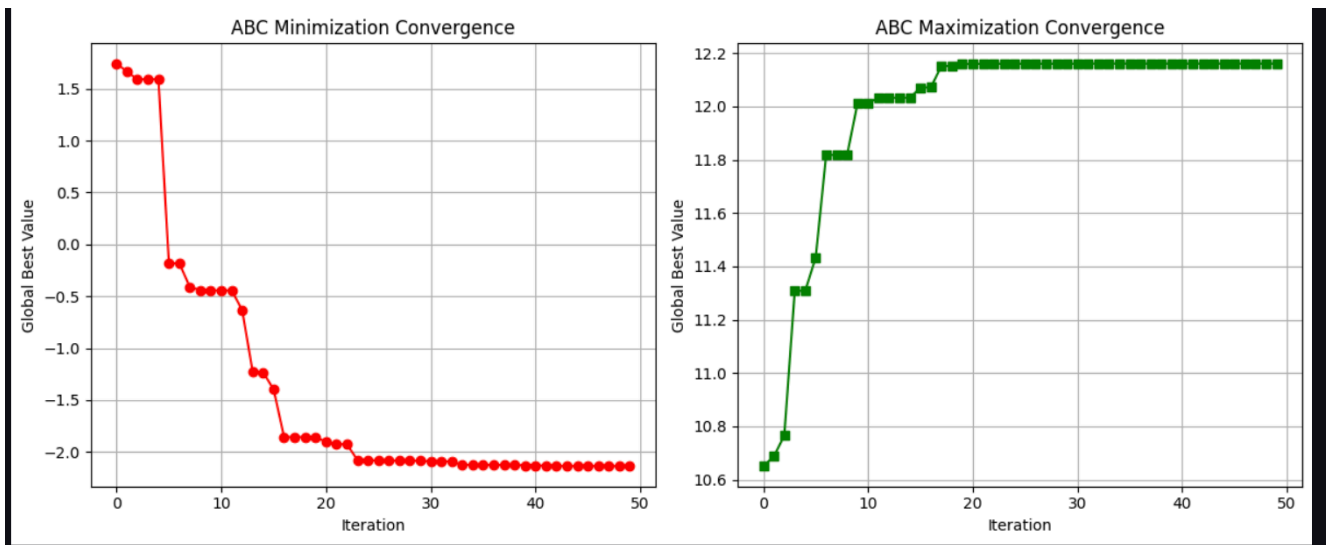


Figure 1 :

Convergence performance tracking for both Minimization and Maximization operations over 50 iterations.

1. Visual Simulation Setup :

We used **NetLogo Web** to build an agent-based simulation model mapping the exact logic of the ABC algorithm.

- **Yellow Circles (Food Sources):** Represent the positions of the 4 decision variables being updated dynamically in the search space.
- **Red Circles (Scout Bees Phase):** Show the exact moment the `limit` threshold (20 failed trials) is breached, forcing the abandonment of poor solutions and launching global random re-exploration.

2. Simulation Screenshot :



Figure 2 : NetLogo visual simulation output showcasing convergence at the 50th tick.

3 . Simulation netlogo Code :

```

1 breed [bees bee]
2 breed [foods food]
3
4 bees-own [ role my-food ]
5 foods-own [ x1 x2 x3 x4 obj-score fitness trial-count ]
6
7 to setup
8   clear-all
9   reset-ticks
10  create-foods 15 [
11    set xcor random-xcor set ycor random-ycor
12    set shape "circle" set color yellow
13    set x1 (random-float 10 - 5) set x2 (random-float 10 - 5)
14    set x3 (random-float 10 - 5) set x4 (random-float 10 - 5)
15    set trial-count 0
16    calculate-my-score
17  ]
18  create-bees 15 [
19    set shape "bug" set color orange set role "employed"
20    set my-food one-of foods move-to my-food
21  ]
22 end
23
24 to calculate-my-score
25   set obj-score (abs x1 + abs x2 + abs x3 + abs x4)
26   set fitness (1 / (1 + obj-score))
27 end
28
29 to go
30   ask bees [
31     let old-fit [fitness] of my-food
32     ask my-food [
33       set x1 (x1 + random-float 0.2 - 0.1)
34       calculate-my-score
35       if fitness <= old-fit [ set trial-count (trial-count + 1) ]
36     ]
37   ]
38   ask foods [
39     if trial-count > 20 [
40       set x1 (random-float 10 - 5) set x2 (random-float 10 - 5)
41       set trial-count 0 calculate-my-score set color red
42     ]
43   ]
44   tick
45   if ticks >= 50 [ stop ]
46 end

```

4 . Traffic Basic Settings for Netlogo simulation

powered by NetLogo

Traffic Basic
Modified from original

File:

Export:

Mode: Interactive Commands and Code: Bottom

Model Speed ticks: 50

number-of-cars

setup

acceleration

deceleration

Car speeds

speed

time